

How Uber Built its Observability Platform

Uber built an in-house observability platform called M3 to cope with huge quantities of data, as the business grew with breackneck speed. Martin Mao, former manager on the team, shares the story.



GERGELY OROSZ

SEP 12, 2023 • PAID



39



2



2

Share



The ride-hailing service's open-source, large-scale metrics engine called [M3](#), is an impressive piece of software. When I worked at Uber, my team – Rider Payments – was among the first to onboard this new and internal platform in 2017. We needed to monitor payments business metrics across hundreds of major cities, and get alerted when any payment method saw regressions on either iOS or Android: and M3 was built to support such use cases.

[Martin Mao](#) headed up M3's engineering team at the time, from the writing of the first line of code for it, until it was rolled out across the organization. He's since left Uber to co-found [Chronosphere](#), an efficient observability platform. I caught up with Martin, who believes the current production version of M3 is probably *still* one of the largest in the world, scale-wise. In 2018, it processed 600 million data points per second (!!), and this has only grown since.

I asked Martin how they built M3, and what he learned from leading that effort. Today, we cover:

1. Why Uber needed a new observability platform
2. How M3 was born
3. Rolling out M3 across the company
4. Engineering challenges
5. The evolution of M3, 2016-2019

6. Becoming manager of the observability team
7. Learnings from building an observability product and team from scratch

With that, it's over to Martin, who narrates the rest of this article.

1. Why Uber needed a new observability platform

In 2015 when I started at Uber, the observability platform was a [Graphite](#), [Carbon](#) and [WhisperDB](#) stack, which had several issues:

- **Not horizontally scalable:** it was not possible to add capacity to the system just by adding more machines
- **No replicas:** if a node died, the company lost data. There was only a handful of nodes, so one node dying meant the loss of around 1/8th of all Uber's data!
- **Adding capacity took too long:** the system needed to be taken offline for a week or more, to add capacity

During the first week I spent oncall, all I did was delete data from the backend to free up much-needed space and keep the observability stack up and running.

In order to try and fix these issues, we built a better observability platform from scratch in around 6 months, using these technologies:

- **Cassandra:** for the time series database
- **ElasticSearch:** for the metrics index
- **Go:** for all the custom code we wrote

We managed to stand up the new system in time for Halloween, 2015, which was Uber's second largest peak load event of the year, behind New Year's Eve. That year was the first time Uber's observability system did not have an outage during the Halloween peak!

At the time, Uber was growing 10x year-on-year in data volume. We could barely keep up, and kept adding Cassandra and ElasticSearch clusters, pushing both technologies to their limits. At one point, we operated around 700 Cassandra nodes! As

a team, we knew we couldn't keep adding node upon node, long-term, We needed a more scalable approach.

We pitched a project to leadership to do a rewrite of the observability stack, arguing that the observability stack we'd just put in place with Cassandra was just a quick workaround. We argued that the only sensible option was to build a new one, if Uber wanted to continue having visibility of:

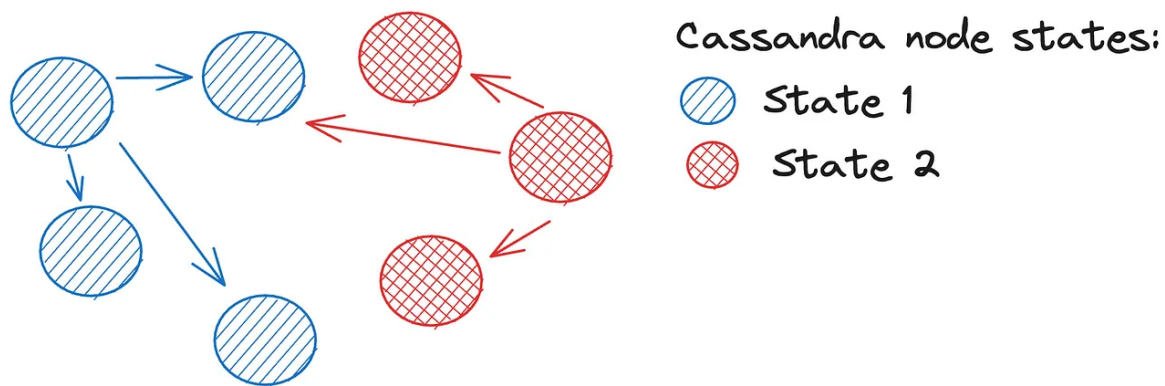
- Infrastructure
- Microservices
- Realtime business metrics like number of rides, amount of payments processed

Buying was not an option because there was no vendor that could handle Uber's scale in terms of data to be processed, at the time.

2. How M3 was born

Split-brain issues were a major reason why we proposed a rewrite. Split-brain occurs when two nodes lose the ability to synchronize, usually as a result of data corruption or other inconsistencies. Back then, we used Cassandra as a time-series database, even though it was built as a key-value store. In hindsight, we didn't choose the right tool for the job, and this was made worse by how frequently we suffered networking issues at [data centers we operated](#):

The split-brain problem



When nodes got out of sync (e.g. thanks to a network problem) then what is the correct state of the network?
What state should be propagated, and treated as correct?

pragmaticengineer.com

The split-brain problem, illustrated. Nodes can get out of sync on what the state of the network is, thanks to networking issues. Once this happens, it's challenging to figure out the source of truth.

The problem we had was that every time there was a networking issue, the amount of split-brains was huge. Cassandra leverages gossip protocol, which means all nodes talk to each other, and spread what each node understands as its view of the world. However, in an environment with many network issues, it's easy to gossip the *wrong* information. This causes a split-brain, where half of a cluster thinks the state is X, while the other half thinks it's Y. It's hard to recover from a split-brain state.

I remember we had to write custom code to detect which half of the cluster had the correct details, and then identify which nodes to automatically shut down, in order to stop the wrong information from propagating.

We wanted to move to a whole new world; one where instead of gossip, we could rely on quorum, and being able to say: "let's have one, consistent five-node cluster, that

has the correct state of the network. If any nodes cannot talk to this cluster, do not assume it's a different view of the world, until you can talk to this cluster."

Due to Uber's scale, we didn't have the option of using a vendor. From everything I know about observability at Uber, I would be surprised if there is a vendor that can handle Uber's load, even today. If there is one, I'd suspect the cost of handling the load would exceed [\\$65M/year](#), if purchased from a vendor.

So, we had to build our own, and "M3" was the name we chose for the project. Here's a little secret I'll let you in on: M3 stands for 'Matt Mihic's Metrics.' [Matt](#) was the original tech lead on the project; and today he's VP of Engineering at Chronosphere. [Mantas Klasavičius](#), from Vilnius in Lithuania, was the second engineer on the team. He's now head of engineering EU at Chronosphere. I was the third member of the team, and I'm the founder and CEO of Chronosphere.

We decided to build M3 [as open source](#) from the start, and to not depend on Cassandra for storage, but to build a custom time series database that would allow M3 to scale better. For our consistency model, we decided to build on top of [etcd](#), a distributed, reliably key-value store.

The technology stack did not change much. We kept using **Go**. A lot of our Go code felt a bit like we'd used C++ though, because we manually managed a lot of memory at the time. Go's [garbage collector](#) wasn't that great, but we chose Go because it's a very readable language that's easy for any new team member to get up to speed with.

3. Rolling out M3 across Uber

By Halloween 2016, the "new" M3 stack was shadowing the "old" stack we'd built in time for the previous Halloween. It's interesting to reflect on how quickly Uber grew: by 2016, the 2015 stack was already "legacy." Still, that "old" stack handled 2016's Halloween peak load, even if we did have to resort to a few hacks to manage observability data that was 10x higher than the previous year. For instance, we kept all recent data in a separate, temporary in-memory data store.

By early 2017, we were confident the new stack, M3, was ready for prime time. We switched over and never looked back. *Read more [about shadowing and migration strategies in this article](#).*

We gradually transitioned the whole company over to M3. The payments team in Amsterdam, the Netherlands, was one of the early customers because it had urgent observability requests around business metrics. Over time, we transitioned all teams within Uber to M3. With each onboarding, we had to add a few use cases.

Areas we migrated included all the infrastructure and network monitoring, all our microservices observability, the mobile apps, and all business operational metrics.

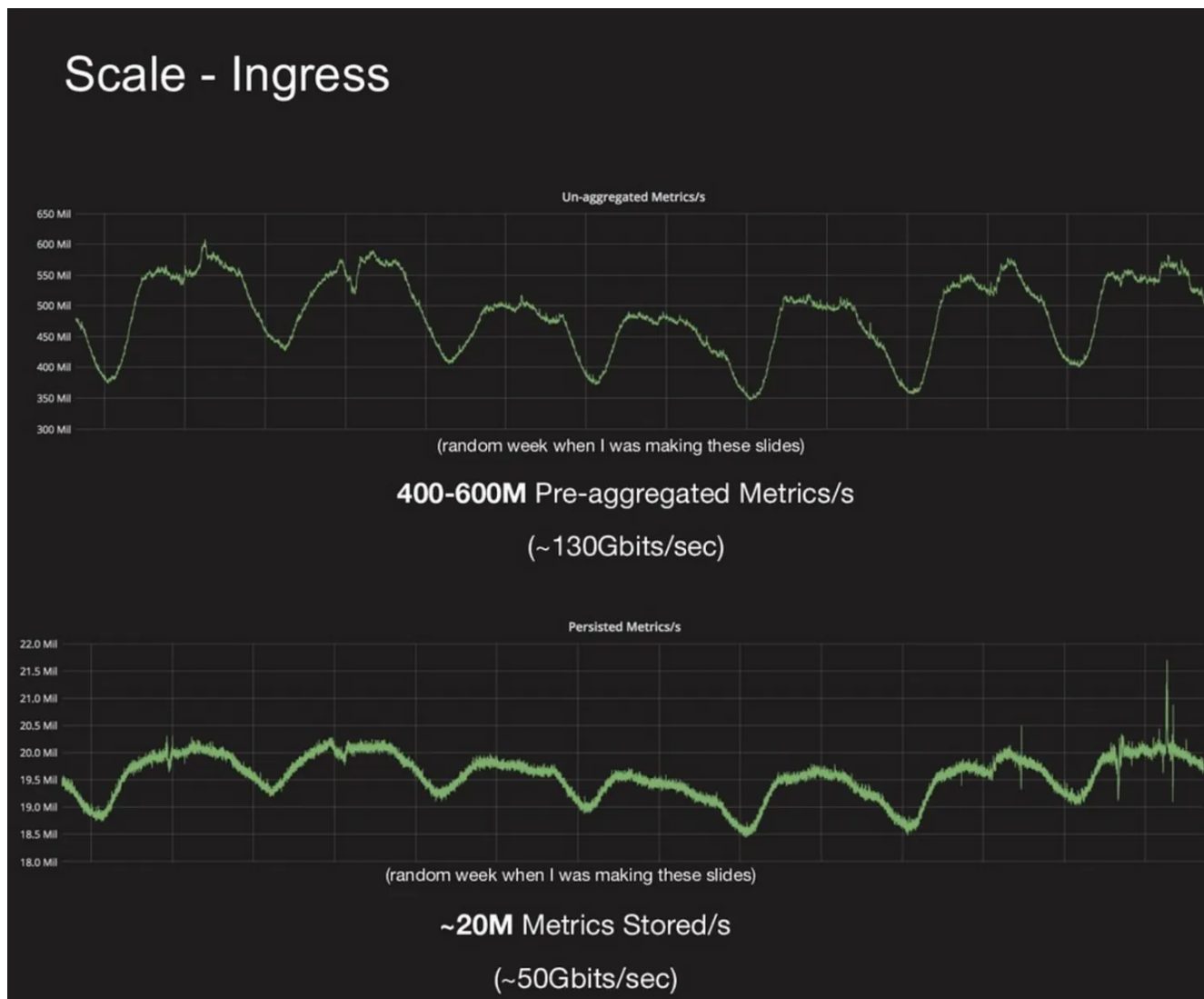
In the first iteration of the product, we 'only' shipped a scalable backend. Eventually, we added a lot more, like a much more complicated anomaly detection called "F3," and automated dashboards. However, early on we focused on having a reliable, performant and cost-efficient backend.

4. Engineering challenges

Scale was a challenge. Back in 2018, M3 had these stats:

Ingress (transfer data in):

- Ingesting [600 million metrics per second](#), requiring ~130 Gbits/sec bandwidth
- Storing 20 million resulting metrics per second, requiring ~50 Gbits/sec bandwidth



The ingress scale handled by M3, in 2018. Image source: [Monitorama conference](#)

Egress (transfer data out):

- Getting 2,000 queries/second
- Powering 9,000 Grafana dashboards and 150,000 realtime alerts
- Fetching ~30B data points per second, requiring ~20Gbits/sec bandwidth

After the full rollout, we found 98% of Uber's developers – ~4,000 at peak – used M3 in some way.

High cardinality was a major challenge. Cardinality refers to the uniqueness of data values in a label or tag. High cardinality means a label has a large number of unique

values, which can be challenging for data processing and analysis. For example, the following use cases are all high cardinality:

- City identifiers at Uber. The company has more than 10,000 unique cities.
- A combination of type of products, payments, and regions. Uber “only” had a few dozen products, a few dozen payment methods, and a couple of regions, but the combination of these produced cardinality in the range of thousands.
- Other examples – which we kept at bay at Uber – include user IDs on a large online platform; or IP addresses in web logs for a service seeing a high number of visitors

And here are examples of low cardinality use cases:

- Top-level categories in an online store: there are usually no more than a few dozen categories for most stores, even those stocking millions of unique items
- Days of the week: these have 7 distinct values, which is low cardinality

When we switched from our old stack to M3, we introduced the concept of tags for teams to use as unique values. Most teams made reasonable use of tags, keeping them as low cardinality. However, the nature of Uber’s business meant that legitimate use cases had high cardinality. For example, take this use case:

- A city label where a ride takes place (Uber operates in ~10,000 cities)
- Plus the product used (Uber had close to a dozen products: like UberX, UberBlack, UberVan, etc.)
- Plus the type of payment (Uber supported ~15-20 different payment methods)
- Plus the region of the request

All of these quickly add up to high cardinality. The more fields are added, the higher the cardinality, and the more challenging it is for the system to process and index these values.

Uber’s dynamic business growth was also a headache. There aren’t many systems around that are designed to handle 10x year-on-year data growth! Two years of consecutive growth means a 100x increase in data stored, which means you need to

redesign the system. With this level of growth, horizontal scaling of a system goes out the window.

To make things more challenging, at Halloween and on New Year's Eve, we'd have about 4x the average daily load, but never got a 4x budget to put 4x as many machines in place for those days. We had to make do with the resources we had. And we *had* to be up because we were the observability team! If we went down, all of Uber would be flying blind.

Storing of data was an interesting challenge. We used raw SSDs to store data, using the custom M3 data file format. We ended up not using [HDFS](#) or [S3](#) as storage like many teams at Uber did, because we found raw file storage was more efficient, at the time.

On the logging side, we produced about 1PB of raw logs per day in around 2018, that were stored in Uber's [log analytics platform](#), built on top of Clickhouse. This data comprised a big chunk of Uber's logs.

5. The evolution of M3

We evolved from focusing on the backend, to focusing on the product itself. When I joined, the most pressing problem was to handle the load coming our way, and to make the backend reliable. By 2017, we'd completed this task, which was the point when we could turn our attention to building these features on top of the platform:

- Better alerting
- Dynamic and composable dashboards
- Anomaly detection
- Tracing and logs

Incidentally, these dashboards eventually evolved into a realtime, global view of our business. It turns out that for Uber, local factors like weather, sporting events, etc., can cause massive local increases in demand. Meanwhile, events like heavy traffic and road closures can dampen demand. These real world factors translated to requests to our

app, and our observability dashboards ended up being one of the primary ways Uber's CEO tracked the business.

The observability team grew to support the widening scope. When I joined in 2015, the team was 3 people. When I left 4 years later, the observability team had grown to 75 people. These are the teams we created during this time, comprising 10-25 people:

- M3 team
- Tracing team
- Anomaly detection team. This team built the "F3" tool.
- Synthetics team: building a black box-testing service
- Observability SRE team
- Application team: building a tool called uMonitor, and dashboarding tools

Uber was unique: we built everything and we also operated it. I would advise most companies against doing this, especially building your observability solution from the ground up, like we did. In general, I see much smaller observability teams in the industry than we had at Uber.

There was a point where we felt our mission was accomplished. Uber's growth slowed, starting from around 2017. The 10x year-on-year growth shrunk to something like 30% annually.

With this kind of growth, you can simply scale horizontally by adding more machines to your system, and those machines soak up the additional load. A slowdown in growth meant we could focus on building out the product features.

As manager of the team, it was more challenging to find 'hard' problems for engineers to solve. This meant it was harder for engineers to get promoted, as solving challenging problems is always a clear path to promotion.

I started to see senior engineers on the team move on to other projects. And they were right to do so; Uber needed them to solve hard problems in other parts of the company! By moving off the observability team, these engineers made room for more junior

engineers who took on now-reduced projects. These engineers still got valuable experience, and many got promoted to the next level because they were not easy problems. But they were simply not as hard as during the very high-growth years.

6. Becoming manager of the observability team

My path to engineering manager on the observability team was this:

- Individual contributor on the team (tech lead)
- Tech lead manager
- Engineering manager

I was one of the pilot employees for Uber's Tech Lead Manager (TLM) program, rolled out in 2017. Before then, the M3 team had had no 'formal' manager since being founded in 2015.

It's hard to find a manager for a very technical team. They turned to me and asked: "hey Martin, do you want to be a tech lead manager?" I was like, "sure, why not." I only realized later that this TLM role wasn't properly specified. I remember at review time they told me I was going to be assessed both as a manager and individual contributor, even though I had a team of 20 engineers! *Note from Gergely: this chimes with what we caution against doing in the article, [Engineering skillset overlaps](#).*

Looking back, I think nobody really thought about how to assess TLMs, before introducing this role. It was terrible; my typical day was back-to-back meetings, with 1:1s and stakeholder meetings. I used to also be on call. It was only at night that I could finally code and I was merely surviving; it was a really tough time.

Over time, the amount of code I wrote started to shrink and I did more code reviews. Finally, my manager said: "hey, we're going to make you an engineering manager."

Being "promoted" to engineering manager became a leveling issue. I was at the 5B level, which is staff engineer today (after [Uber's leveling changes in 2022](#),) and was Senior Engineer 2, back then. When I became a TLM, I got to keep my 5B level.

However, by becoming an engineering manager, I should have moved down to the 5A level, which was the entry-level for engineering managers.

I didn't want to take a step-down in levels to keep doing what I was doing. I also couldn't go back to being an individual contributor, because our team needed the manager role I was filling. In the end, I had to go through an accelerated promotion path, to get promoted from 5A to 5B. My promotion package was basically: "Look, this guy is managing 20 people, and here is all the impact he is driving, can we acknowledge he is at the 5B manager level?"

By the time I became an engineering manager, my role was less technical. My main responsibilities were:

- Getting team members appropriately recognized and promoted at the right time
- Finding engineers relevant challenges that matched our roadmap. Sometimes, this meant tweaking the roadmap to optimize for the team
- Meeting stakeholders and customer teams to ensure they were satisfied with the platform

I had to also play the product manager (PM) role, as we didn't have one. This meant it was also my job to:

- Gather requirements
- Figure out our roadmap
- Prioritize work
- ... and do all the other EM duties

To be fair, all of this was easier to do as an engineering manager for a "pure platform" team, due to a few factors:

- We had direct access to our customers, who were developers at Uber
- Our customers talked with us all the time – via direct inbound pings
- The problem space was known and observability is a well-understood space
- Our goals were clear from the start

For product teams, the above factors tend to be absent, which is why I see the PM role as essential on product teams; it enables the EM to focus more on execution and team processes.

Towards the end of my time, the observability team onboarded a technical program manager (TPM) as we crossed more than 50 engineers in the team. *Coincidentally, Sophia Vincent, the former Director of TPM at Uber, [previously discussed](#) how she thought a 50:1 engineer:TPM ratio tended to be her target.*

7. Learnings from building an observability product and team from scratch

Here are the learnings I took from building the M3 product and team from the ground up.

Starting with a senior team made sense for such a challenging problem space.

When we looked at the original problem space of building a reliable observability solution that could cope with Uber's rapid growth, we knew we needed really senior engineers.

Matt, the first lead of the team, was one of 4 senior staff engineers at Uber. When we started the team, all other software engineers were at the senior-or-above level. We needed this level of expertise to move quickly and build a new database engine in just a year.

The more stable a platform, the easier it is to onboard less senior engineers. As the senior folks completed the challenging tasks – and often got promoted and moved onto other projects – the platform became more stable. With this foundation, we could onboard more SDE 2s (software engineers below senior level.) These people still found the work challenging and worth getting excited about, while the scope of the work was enough for them to get promoted to the senior level.

We consistently struggled to onboard graduate engineers to our team, though. This was because Uber's observability system was – and I believe, still is – one of the largest production monitoring systems in the world. All engineers on the team have to be

oncall, but it's challenging to go oncall without at least a few years' experience operating somewhat similar production systems.

Career progression narrows as a platform becomes more mature. There were major challenges in building the platform and a ton of work to be done, so there was plenty of opportunity and scope for people to pick up tasks which helped them get promoted to staff-or-above levels. Yet, as the platform became more stable, finding such impactful opportunities and scope became harder.

As a manager, you might think you can introduce scope to make promotions to the staff level possible. But this is a bad approach; it's reckless to introduce major changes to a large platform, just to create such scope. Not to mention that it's inefficient from a business perspective.

As a team and product mature, the engineering manager needs to understand the roadmap, and *build their team to execute on the roadmap*, as opposed to building the roadmap to help team members progress to the next levels.

Takeaways

This is Gergely again.

Thank you, Martin, for all these details. You can follow Martin [on LinkedIn](#) and X, formerly [Twitter](#). If you'd like to learn more about M3, I suggest the following engineering resources:

- [M3](#): the project page
- [Introducing M3](#) on Uber's engineering blog
- [Optimizing M3](#) on Uber's engineering blog

A few things stand out for me from Martin's account.

Starting a team with only senior engineers pays off for hard problems. It was interesting to hear how the observability team at Uber did not hire new grads, or even mid-level engineers, for quite some time. They found that to move fast they needed

senior engineers. This made sense, given the problem they had to solve was both hard and impactful. The team's initial goal was to stop observability from going down during Uber's second-busiest day of the year. And they succeeded with this approach!

Planning for extreme growth is hard, and perhaps unnecessary. The first observability solution Martin and the team put in place was based on Cassandra. It was only after building the solution that they realized they'd chosen the wrong tool for the scale which Uber faced, and the problems of the gossip protocol became apparent.

But was it really a mistake to start out with a technology that seemed good enough at the time? I would argue this was not a misstep! If Uber's growth rate was not 10x, year-on-year, then the Cassandra solution would have surely worked for several years. And it would surely be overkill to start by building a custom storage system.

Cardinality is so important to understand and work with, for observability systems. My team onboarded to M3, and we very quickly ran into the problem of cardinality. We wanted to send our logs as they were, and have the M3 team create alerts for us. However, they told us our cardinality was too high for their system to handle, and so it was our job to reduce cardinality – i.e., data variance – if we wanted usable alerts.

The constraint on cardinality meant our engineering team had to review what we really wanted to be alerted for, and how we could reduce the variance in logged data sent to an observability system.

Neglecting cardinality is what tends to create very high observability bills from third-party vendors which can handle high cardinality. They charge according to resource usage, which means charging a lot!

Managing platform teams is a tricky area. It was interesting to hear how the observability team operated for so long without a manager, and how it did not have a product manager, even later on. Martin mentioned it's hard to hire a manager for a very technical, very senior team. When a manager was put in place, it was an existing team member – Martin – who was asked to take the role.

This is a phenomenon I see, again and again. Platform teams with senior engineers are common places for a tech lead manager role to be deployed to. As long as the TLM doesn't have many reports, this approach can work. In Martin's case, 20 reports felt a bit too much!

In a future issue, we will talk with Martin about why he left Uber to build [Chronosphere](#), and his advice for readers planning to work at an infrastructure startup, or to found one.



39 Likes · 2 Restacks

2 Comments



Write a comment...



Brijesh Shukla Sep 13

Details are mostly missing like how it was build it say ETCD + RAW-Logs + Custom File + SSD.

It mostly towards the team & I and Why something was written from scratch not How.

♡ LIKE 💬 REPLY ↗ SHARE

...



Sam Mingo Sep 12

Can we include any numbers around the number of servers for the various components? ETCD, Clickhouse etc.?

♡ LIKE 💬 REPLY ↗ SHARE

...

© 2023 Gergely Orosz · [Privacy](#) · [Terms](#) · [Collection notice](#)
[Substack](#) is the home for great writing